

# GITHUB

## Professional Workflows for Engineers by Sergino Bradford GOUNOUKPEROU

*Branches · Pull Requests · Code Review · Actions · CI/CD · Security*

|               |                                                         |
|---------------|---------------------------------------------------------|
| <b>Part 1</b> | <b>Git &amp; GitHub Fundamentals — The Mental Model</b> |
| <b>Part 2</b> | <b>Branching Strategies — Working in a Team</b>         |
| <b>Part 3</b> | <b>Pull Requests &amp; Code Review</b>                  |
| <b>Part 4</b> | <b>GitHub Actions — Automating Everything</b>           |
| <b>Part 5</b> | <b>CI/CD — From Commit to Production</b>                |
| <b>Part 6</b> | <b>Security, Secrets &amp; Repository Hygiene</b>       |
| <b>Part 7</b> | <b>The Professional Workflow — Putting It Together</b>  |

Tools: Git, GitHub, YAML

# Introduction & Course Overview

## Why This Course Exists

Almost every engineering student knows how to use Git to commit and push code. Far fewer know how to use GitHub as a professional collaboration platform. The gap between 'git push origin main' and the workflows used at Google, Stripe, or Airbnb is enormous — and it's largely invisible from tutorials.

This course is about closing that gap. By the end, you will understand why professional teams structure their repositories the way they do, how to automate the tedious parts of software delivery, how to catch bugs before they reach production, and how to contribute to a codebase in a way that makes your teammates trust and value your work.

## Prerequisites

- Basic Git literacy: init, add, commit, push, pull
- Familiarity with the command line
- A GitHub account

# Git & GitHub

*The Mental Model — What Is Actually Happening*

## Git ≠ GitHub

This distinction matters more than it seems. Git is a distributed version control system — a command-line tool that lives entirely on your machine and manages a directed acyclic graph (DAG) of commits. GitHub is a cloud hosting platform for Git repositories, plus a collaboration layer built on top: issues, pull requests, Actions, access control, and more.

You can use Git without GitHub (GitLab, Bitbucket, Forgejo, or just a bare repo on a server). You cannot meaningfully use GitHub without Git. Understanding the separation helps you reason clearly about what happens locally versus remotely.

## The Commit Graph

At its core, a Git repository is a DAG of commit objects. Each commit contains a snapshot of your tracked files (stored as a tree of blobs), a pointer to its parent commit(s), author metadata, a timestamp, and a SHA-1 hash of all the above. The hash is the commit's identity — changing anything in history changes every hash downstream.

```
# Every commit is a node in the graph
# Branches are just named pointers to commits
# HEAD is a pointer to your current branch (or commit)

a1b2c3f ← commit: 'Initial setup'
  |
d4e5f6a ← commit: 'Add auth module'
  |
7b8c9d0 ← commit: 'Fix login bug'    ← main
  |
1e2f3a4 ← commit: 'Add dashboard'    ← feature/dashboard ← HEAD
```

## Local vs. Remote: The Four Zones

One of the most common sources of confusion is understanding where your changes are at any given moment. There are four distinct zones:

| Zone                 | Location               | How you move code here |
|----------------------|------------------------|------------------------|
| Working Directory    | Your filesystem        | Edit files normally    |
| Staging Area (Index) | Local Git object store | git add <files>        |
| Local Repository     | Local Git object store | git commit             |
| Remote Repository    | GitHub servers         | git push / git fetch   |

## The Three Git Objects You Must Know

### Blob

A blob stores the raw content of a file at a given point in time. It has no filename — just content. Two files with identical content share one blob. Blobs are immutable.

### Tree

A tree maps filenames and directory structure to blobs and other trees. It represents the state of a directory at a point in time.

### Commit

A commit points to a root tree (the full snapshot of your project), the parent commit(s), author, message, and timestamp. The SHA-1 hash of all this content is the commit ID.

## Essential Commands You Should Have Memorized

```
# Inspect the state of everything
git status                # what's staged, unstaged, untracked
git log --oneline --graph --all  # visualize the full commit DAG
git diff                  # working dir vs staging
git diff --staged         # staging vs last commit

# Undo things safely
git restore <file>         # discard working dir changes
git restore --staged <file> # unstage without losing changes
git revert <commit>        # create a new commit that undoes a commit

# Inspect history
git show <commit>         # see a commit's diff and metadata
git blame <file>          # see who last changed each line
git log --follow -p -- <file> # full history of a file including renames
```

⚠ *Never use 'git reset --hard' or 'git push --force' on shared branches. These rewrite history and cause chaos for your teammates. We cover when force-pushing is acceptable (and how to do it safely) in Part 2.*

## Merge vs. Rebase — The Core Trade-off

Both merge and rebase integrate changes from one branch into another. They produce equivalent code but different histories, and the choice affects readability and debuggability of your project long-term.

### ✗ Merge: preserves history but creates noise

```
# Merge creates a 'merge commit' — an extra commit with two parents
# In a busy repo, the log looks like a bowl of spaghetti
```

```
git checkout main
git merge feature/login
```

```
# Result in git log --oneline --graph:
* 8f2a1b0 Merge branch 'feature/login' into main
| \
| * 3d4e5f6 Add login form validation
| * 1a2b3c4 Add login route
| /
* 9z8y7x6 Previous work on main
```

### ✓ Rebase: clean linear history – preferred for feature branches

```
# Rebase replays your commits on top of the target branch
# Result: a perfectly linear history, easy to git bisect
```

```
git checkout feature/login
git rebase main
```

```
# Result in git log --oneline:
* 5e6f7a8 Add login form validation ← new SHA (replayed commit)
* 2b3c4d5 Add login route          ← new SHA (replayed commit)
* 9z8y7x6 Previous work on main
```

```
# Then fast-forward merge – no merge commit:
git checkout main && git merge feature/login
```

💡 *The golden rule of rebase: never rebase a branch that others are working on. Rebasing rewrites SHA hashes, which breaks everyone else's local copies. Rebase your private feature branches; merge or use PRs for shared ones.*

# Branching Strategies

*How Professional Teams Organize Work*

## Why Branch Strategies Matter

Without a branching convention, every team member develops their own habits. The result is a repository where no one knows which branch is deployable, hotfixes interfere with features in progress, and releasing new versions requires manual cherry-picking. A branching strategy is a shared contract that answers three questions: where does deployable code live, how does new work enter the codebase, and how do we handle urgent production fixes?

## Strategy 1: Git Flow

Git Flow, introduced by Vincent Driessen in 2010, uses two permanent branches and several types of short-lived ones. It was designed for projects with scheduled releases and long-lived versions.

| Branch           | Purpose                                                  | Merges into       |
|------------------|----------------------------------------------------------|-------------------|
| <b>main</b>      | Production-ready code only. Tagged with version numbers. | —                 |
| <b>develop</b>   | Integration branch. All completed features land here.    | main (at release) |
| <b>feature/*</b> | One branch per feature. Branched from develop.           | develop           |
| <b>release/*</b> | Stabilization. Bug fixes only, no new features.          | main + develop    |
| <b>hotfix/*</b>  | Emergency production fixes. Branched from main.          | main + develop    |

```
# Git Flow in practice
git checkout develop
git checkout -b feature/user-authentication

# ... do work, commit ...

# When feature is complete:
git checkout develop
git merge --no-ff feature/user-authentication
git branch -d feature/user-authentication

# Prepare a release:
git checkout -b release/1.2.0 develop
# ... fix bugs, bump version ...
git checkout main && git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m 'Release 1.2.0'
git checkout develop && git merge --no-ff release/1.2.0
```

⚠ *Git Flow is powerful but heavy. It adds ceremony that slows down teams that deploy multiple times per day. It shines for versioned libraries, desktop software, or mobile apps with release cycles.*

## Strategy 2: GitHub Flow

GitHub Flow is dramatically simpler and designed for teams that deploy continuously. There is one permanent branch (main, always deployable) and short-lived feature branches. Everything goes through a Pull Request.

### ✓ GitHub Flow – the professional default for web services

```
# 1. Create a branch from main
git checkout main && git pull
git checkout -b feature/add-dark-mode

# 2. Make commits
git add . && git commit -m 'feat: add dark mode toggle'
git push -u origin feature/add-dark-mode

# 3. Open a Pull Request on GitHub
# → Automated checks run (tests, linting, build)
# → Team reviews code
# → PR is approved and merged to main

# 4. main is immediately deployable (and usually auto-deployed)

# 5. Delete the feature branch
git branch -d feature/add-dark-mode
```

## Strategy 3: Trunk-Based Development

Trunk-Based Development (TBD) takes simplicity to its logical extreme: all developers commit directly to main (the 'trunk') multiple times per day. Feature branches exist for at most one or two days. Incomplete features are hidden behind feature flags in code, not in branches.

This strategy requires excellent CI (tests must run in seconds and be comprehensive), mature feature flagging infrastructure, and a high-trust team. It is the approach used internally at Google and Meta for their largest codebases. It eliminates merge conflicts almost entirely because branches are so short-lived.

## Branch Naming Conventions

Consistent branch names are machine-readable, letting automation know what a branch is for. The most widely adopted convention:

| Branch prefix   | Meaning & example                            |
|-----------------|----------------------------------------------|
| <b>feature/</b> | New functionality: feature/user-profile-page |
| <b>fix/</b>     | Bug fix: fix/login-redirect-loop             |

|                  |                                                            |
|------------------|------------------------------------------------------------|
| <b>hotfix/</b>   | Urgent production fix: hotfix/payment-null-pointer         |
| <b>chore/</b>    | Maintenance, deps, config: chore/upgrade-node-18           |
| <b>docs/</b>     | Documentation only: docs/update-api-reference              |
| <b>refactor/</b> | Code improvement, no behavior change: refactor/auth-module |
| <b>test/</b>     | Adding or fixing tests: test/add-checkout-coverage         |

## Commit Message Conventions — Conventional Commits

Conventional Commits is a specification that gives commit messages a machine-parseable structure. Tools can then automatically generate changelogs, determine semantic version bumps, and filter CI runs based on commit types.

```
# Format: <type>(<scope>): <description>
#
# type: feat | fix | docs | style | refactor | test | chore | ci | perf
# scope: optional, what component changed
# description: imperative mood, lowercase, no period

feat(auth): add OAuth2 login with Google
fix(cart): prevent negative quantity on item removal
docs(api): add rate limiting section to README
refactor(db): extract connection pool to separate module
test(payment): add edge cases for expired card handling
chore(deps): upgrade axios from 0.27 to 1.6
ci(actions): add caching for node_modules

# Breaking changes — add ! or BREAKING CHANGE footer:
feat(api)!: rename /users endpoint to /accounts
```

💡 Tools like 'commitlint' can enforce Conventional Commits in a pre-commit hook or CI check, ensuring the whole team follows the convention consistently.

# Pull Requests & Code Review

*The Heart of Collaborative Development*

## What a Pull Request Actually Is

A Pull Request (PR) is a proposal to merge one branch into another. But thinking of it purely as a merge mechanism misses the point. A PR is primarily a communication tool: it creates a structured conversation around a change, records the reasoning behind decisions, triggers automated checks, and creates an audit trail of every modification to the codebase.

In professional teams, the quality of your PRs is how you build reputation. A well-structured PR that's easy to review, includes context, and passes all checks is a signal of seniority and craftsmanship. A 3,000-line PR with the message 'misc fixes' is a red flag.

## Anatomy of a Great Pull Request

- Title: concise, follows Conventional Commits format. 'feat(dashboard): add real-time metrics chart'
- Description: explains WHY the change exists, not just what it does. Links to the issue or ticket it solves.
- Screenshots or recordings: for any UI change, include before/after visuals.
- Testing notes: what did you test manually? What automated tests cover this?
- Breaking changes: explicitly call out anything that changes public APIs or requires migration.
- Size: ideally under 400 lines changed. Large PRs are hard to review well and introduce more bugs.

### ✗ A PR no one wants to review

Title: fixes

Description: (empty)

Files changed: 47 files, +2847 lines, -1203 lines

### ✓ A PR that gets fast, quality reviews

Title: feat(auth): add two-factor authentication via TOTP

Description:  
Closes #142

Adds TOTP-based 2FA using the speakeasy library.  
Users can now enable 2FA from their account settings.  
A QR code is generated on setup and verified on each login.

Testing:  
- Unit tests for TOTP generation/validation in auth.test.ts

```
- Manual test: enrolled on staging, confirmed works with Google Auth
```

```
Breaking changes: none
```

```
Files changed: 8 files, +312 lines, -14 lines
```

## Doing Code Review Well

Code review is a skill that takes years to develop. The goal is not to catch every imperfection — it's to ensure correctness, maintainability, and knowledge sharing. Here are the most important principles:

### As a Reviewer

- Review the code, not the person. 'This function could be simplified' not 'you wrote this wrong'.
- Be explicit about severity. Use prefixes: 'nit:' for minor style, 'suggestion:' for optional improvements, 'blocker:' for things that must change before merge.
- Explain your reasoning. 'nit: consider extracting this into a helper — it's called in three places' is much more useful than 'nit: refactor this'.
- Ask questions rather than making assumptions. 'Why was this approach chosen over X?' opens a productive conversation.
- Approve explicitly when satisfied. Don't leave reviews in limbo.

### As a PR Author

- Respond to every comment, even if just to acknowledge ('done' or 'discussed in call').
- Don't take blocking feedback personally. The goal is a better codebase.
- Resolve threads after addressing them so reviewers know what still needs attention.
- Avoid force-pushing to a branch under review — it makes it impossible to see what changed since the last review.

## Branch Protection Rules

Branch protection rules are settings on GitHub that enforce your team's workflow. For any production branch (main, develop), you should enable:

| Rule                                 | What it prevents                                     |
|--------------------------------------|------------------------------------------------------|
| <b>Require pull request reviews</b>  | No one can merge without at least 1 (or N) approvals |
| <b>Require status checks to pass</b> | Broken tests or failed builds cannot be merged       |

|                                          |                                                             |
|------------------------------------------|-------------------------------------------------------------|
| <b>Require branches to be up to date</b> | PRs must be rebased/merged with main before merging         |
| <b>Require linear history</b>            | Enforces rebase strategy, keeps history clean               |
| <b>Restrict who can push</b>             | Prevents direct pushes to protected branches                |
| <b>Require signed commits</b>            | Ensures commits are cryptographically attributed to authors |

💡 *Set branch protection rules before you invite collaborators — it's much harder to enforce conventions retroactively than to establish them from the start.*

## Merge Strategies

GitHub offers three ways to merge a PR. Each produces a different history:

| Strategy                  | What it does                                       | Use when                                                |
|---------------------------|----------------------------------------------------|---------------------------------------------------------|
| <b>Merge commit</b>       | Adds a merge commit, preserves full branch history | You want the full context of the branch visible         |
| <b>Squash &amp; merge</b> | Combines all PR commits into one clean commit      | Feature branches with messy WIP commits                 |
| <b>Rebase &amp; merge</b> | Replays commits linearly, no merge commit          | Clean, atomic commits you want to preserve individually |

# GitHub Actions

*Automating Everything in Your Repository*

## What GitHub Actions Is

GitHub Actions is a platform for automating software workflows directly in your repository. A workflow is a YAML file in the `.github/workflows/` directory that describes: when to run (events), what to run on (runners), and what to run (jobs and steps). Actions can run on every push, on a schedule, when an issue is labeled, when a PR is opened — the trigger system is extremely flexible.

Actions replaced a whole ecosystem of external tools: Jenkins, Travis CI, CircleCI (for many use cases), manual release scripts, cron jobs. The key advantage is deep GitHub integration — Actions can read repository context, post comments, create releases, and manage labels natively.

## Core Concepts

| Concept         | Definition                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------|
| <b>Workflow</b> | A YAML file defining automation. Lives in <code>.github/workflows/</code>                     |
| <b>Event</b>    | What triggers the workflow: push, pull_request, schedule, workflow_dispatch, etc.             |
| <b>Job</b>      | A set of steps that run on the same runner. Jobs run in parallel by default.                  |
| <b>Step</b>     | A single task: run a shell command or use a pre-built Action                                  |
| <b>Runner</b>   | The machine that executes jobs: GitHub-hosted (ubuntu, windows, macos) or self-hosted         |
| <b>Action</b>   | A reusable unit of automation, published to the GitHub Marketplace                            |
| <b>Artifact</b> | Files produced by a workflow (build outputs, test reports) stored for later steps or download |
| <b>Secret</b>   | Encrypted values (API keys, passwords) injected as environment variables at runtime           |

## Your First Workflow: Run Tests on Every PR

```
✓ Basic CI workflow — .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [ main, develop ]
```

```

pull_request:
  branches: [ main, develop ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'           # cache node_modules between runs

      - name: Install dependencies
        run: npm ci             # deterministic install from package-lock.json

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build

```

## Key Workflow Features

### Matrix Builds — Test Across Multiple Environments

Matrices let you run the same job across a combination of values — multiple Node versions, OS targets, or compiler flags — without duplicating YAML:

```

jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node: ['18', '20', '22']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm ci && npm test
    # Produces 9 parallel jobs (3 OS × 3 Node versions)

```

### Job Dependencies and Conditional Steps

```

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npm test

  deploy:
    needs: test          # only runs if 'test' job succeeds
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' # only on main branch
    steps:
      - run: ./deploy.sh

```

## Caching Dependencies

Without caching, every workflow run downloads dependencies from scratch. On a large project, this wastes minutes per run and costs money on GitHub's billable minutes:

```

- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-{{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-
# The cache key changes when package-lock.json changes,
# forcing a fresh install when dependencies are updated.

```

## Uploading Artifacts

```

- name: Run tests with coverage
  run: npm test -- --coverage

- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: coverage/
    retention-days: 7
# Coverage report is downloadable from the Actions run page

```

 The GitHub Marketplace has thousands of community Actions: send Slack notifications, post Lighthouse scores on PRs, auto-assign reviewers, generate release notes, deploy to AWS/GCP/Azure. Before writing a custom script, check if an Action already exists.

# CI/CD Pipelines

*From Every Commit to Production*

## Continuous Integration vs. Continuous Delivery vs. Continuous Deployment

These three terms are often conflated. They represent three levels of automation in the software delivery pipeline, each building on the previous:

| Practice                           | What it means                                                                          | Human involvement         |
|------------------------------------|----------------------------------------------------------------------------------------|---------------------------|
| <b>Continuous Integration (CI)</b> | Every commit is automatically built and tested. Broken builds are caught immediately.  | None — fully automated    |
| <b>Continuous Delivery (CD)</b>    | Every passing build is automatically packaged and staged, ready to deploy at any time. | Manual approval to deploy |
| <b>Continuous Deployment (CD)</b>  | Every passing build is automatically deployed to production with no human gate.        | None — fully automated    |

Most mature teams practice CI + Continuous Delivery, with a manual approval gate for production deployments. Fully automated deployment to production (Continuous Deployment) is common in high-trust teams with excellent test coverage and robust rollback mechanisms.

## A Complete CI/CD Pipeline with GitHub Actions

Here is a realistic, production-grade pipeline for a Node.js web service. It runs on every PR and on every push to main:

```
name: CI/CD Pipeline

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

jobs:
  # — Stage 1: Quality Gates —————
  quality:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with: { node-version: '20', cache: 'npm' }
- run: npm ci
- run: npm run lint
- run: npm run type-check
- run: npm test -- --coverage
- uses: actions/upload-artifact@v4
  with: { name: coverage, path: coverage/ }

# — Stage 2: Build Docker Image —————
build:
  needs: quality
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
  outputs:
    image-tag: ${ steps.meta.outputs.tags }
  steps:
    - uses: actions/checkout@v4
    - name: Log in to container registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
    - name: Build and push image
      uses: docker/build-push-action@v5
      with:
        push: ${ github.ref == 'refs/heads/main' }
        tags: ${ steps.meta.outputs.tags }

# — Stage 3: Deploy (main branch only) —————
deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://myapp.example.com
  steps:
    - name: Deploy to server
      run: |
        curl -X POST ${ secrets.DEPLOY_WEBHOOK } \
          -H 'Authorization: Bearer ${ secrets.DEPLOY_TOKEN }'

```

## Environments and Deployment Gates

GitHub Environments let you define deployment targets (staging, production) with protection rules. You can require manual approvals before a job runs in an environment, restrict which branches can deploy to it, and track deployment history. This is how you implement a human gate in a Continuous Delivery pipeline:

```
# In repository Settings → Environments → production:
# - Required reviewers: [senior-dev-1, senior-dev-2]
# - Wait timer: 5 minutes (gives time to cancel)
# - Deployment branches: main only

# In the workflow:
deploy-production:
  environment:
    name: production      # triggers the protection rules
    url: https://app.example.com
  # This job will PAUSE until a required reviewer approves it on GitHub
```

💡 *Always deploy to a staging environment first and run smoke tests before promoting to production. A two-stage deploy (staging → production) catches environment-specific bugs before they affect real users.*

# Security, Secrets & Repository Hygiene

*What You Must Never Get Wrong*

## Never Commit Secrets

The most common and most catastrophic security mistake in student (and professional) projects is committing API keys, passwords, tokens, or certificates to a repository. Even if you delete the file in the next commit, the secret is permanently in Git history and can be extracted with git log. If the repo is ever public, automated bots scan GitHub continuously and will find and exploit your secrets within minutes.

⚠ *If you commit a secret to any repository — public or private — treat it as compromised. Rotate it immediately before doing anything else.*

### ✗ Never do this – not even in a private repo

```
// config.js
const DB_PASSWORD = 'my_super_secret_password_123';
const STRIPE_KEY = 'sk_live_aBcDeFgHiJkLmNoPqRsTuV';
const AWS_SECRET = 'wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY';
```

### ✓ The correct pattern – environment variables

```
// config.js
const DB_PASSWORD = process.env.DB_PASSWORD;
const STRIPE_KEY = process.env.STRIPE_SECRET_KEY;
const AWS_SECRET = process.env.AWS_SECRET_ACCESS_KEY;

// .env file (local development only – in .gitignore!)
// DB_PASSWORD=my_super_secret_password_123

// GitHub Actions – stored as Repository Secrets,
// accessed as: ${ secrets.DB_PASSWORD }
```

## Essential .gitignore Entries

```
# Secrets and environment
.env
.env.local
.env.*.local
*.pem
*.key
*.p12

# Build artifacts
node_modules/
dist/
build/
```

```

__pycache__/  

*.pyc  

target/          # Rust/Java  

*.o *.a *.so     # C/C++  

  

# Editor and OS noise  

.DS_Store  

Thumbs.db  

.idea/  

.vscode/         # (unless you commit shared settings intentionally)  

  

# Logs and temp files  

*.log  

*.tmp  

coverage/

```

## GitHub Security Features

- Dependabot: automatically opens PRs to update vulnerable dependencies. Enable in Settings → Security.
- Secret scanning: GitHub scans all pushes for known secret patterns (API keys, tokens) and alerts you instantly.
- Code scanning (CodeQL): static analysis that finds security vulnerabilities in your code. Configure as a GitHub Action.
- Security advisories: privately disclose and coordinate fixing vulnerabilities before public disclosure.

## Permissions and Access Control

For team repositories, grant the minimum permissions needed. GitHub repository roles from least to most privileged: Read → Triage → Write → Maintain → Admin. Service accounts used by automation (bots, deploy keys) should have the narrowest possible permissions — never Admin.

 Use `CODEOWNERS` to automatically request reviews from the right people when specific files are changed. Put it at `.github/CODEOWNERS`. Example: `'src/auth/**@security-team'`

# The Professional Workflow

*Putting Everything Together*

## End-to-End: From Issue to Production

Here is the complete lifecycle of a feature in a professional GitHub-based team, integrating everything from this course:

1. Issue created: a bug or feature request is filed on GitHub Issues (or Jira/Linear), with acceptance criteria.
2. Branch: developer creates 'feature/add-payment-retry' from main. Branch is linked to the issue.
3. Commits: developer makes small, atomic commits with Conventional Commit messages.
4. Draft PR: developer opens a draft PR early — not for review, but to trigger CI and share progress.
5. CI runs: GitHub Actions runs linting, tests, type checking, and security scans on every push.
6. PR ready: developer marks PR as ready, writes a thorough description, and requests reviewers.
7. Code review: reviewers leave structured feedback. Automated checks must all pass before merge is allowed.
8. Approval: at least one (or two, for sensitive changes) reviewers approve. Branch protection enforces this.
9. Squash & merge: PR is merged with a clean squash commit to main. Branch is auto-deleted.
10. CD triggers: the push to main triggers the deploy workflow. It builds a Docker image, deploys to staging, runs smoke tests, awaits approval, then deploys to production.
11. Monitoring: deployment is tagged in the observability stack (Datadog, Sentry) so any new errors are correlated with the release.

## The Non-Obvious Lessons

- Small PRs merge faster and have fewer bugs. A PR under 300 lines takes 20 minutes to review. A PR over 1000 lines takes days and reviewers stop paying attention.
- CI is only as good as your tests. 100% passing tests on a project with no tests is meaningless. The goal is confidence that main is always deployable.
- Automation is documentation. A GitHub Actions workflow file tells a new team member exactly how to build, test, and deploy the project — better than any README.
- Git history is a communication tool. Write commit messages for the person debugging this at 2am six months from now. That person is probably you.

- Secrets are not a one-time concern. Build a habit of auditing permissions, rotating credentials regularly, and running dependency updates. Security is a process, not a state.

# Reference Summary

*Quick-reference for everything covered in this course*

## Branching Strategy Cheat Sheet

| Strategy        | Best for                                   | Complexity                      |
|-----------------|--------------------------------------------|---------------------------------|
| GitHub Flow     | Web services, SaaS, continuous deployment  | Low — 1 permanent branch        |
| Git Flow        | Versioned software, scheduled releases     | Medium — 2 permanent branches   |
| Trunk-Based Dev | High-trust teams, mature CI, feature flags | Low complexity, high discipline |

## GitHub Actions Quick Reference

| Event trigger             | Syntax                              |
|---------------------------|-------------------------------------|
| Push to specific branch   | on: push: branches: [main]          |
| Pull request              | on: pull_request: branches: [main]  |
| Manual trigger            | on: workflow_dispatch               |
| Scheduled (cron)          | on: schedule: - cron: '0 9 * * 1'   |
| Another workflow finishes | on: workflow_run: workflows: ['CI'] |

## The Golden Rules

|                                                               |
|---------------------------------------------------------------|
| Never commit secrets. Rotate immediately if you do.           |
| main is always deployable. All work happens in branches.      |
| Small PRs get better reviews. Aim for under 400 lines.        |
| A passing CI is a minimum bar, not a guarantee of quality.    |
| Write commit messages for the engineer debugging this at 2am. |
| Branch protection rules before you invite collaborators.      |

## Going Further

- GitHub Docs — [docs.github.com](https://docs.github.com): complete reference for Actions, branch protection, environments
- 'GitHub Actions in Action' — Manning Publications, a comprehensive deep-dive
- act ([github.com/nektos/act](https://github.com/nektos/act)) — run GitHub Actions locally for faster development iteration
- semantic-release — automate versioning and changelog generation from Conventional Commits
- Sigstore / cosign — sign your container images and artifacts to prove they came from your CI

— *End* —